



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

BYTESTREAK

Absolvent

Trifan Bogdan-Cristian

Coordonator științific

Conf. Dr. Radu Boriga

București, iunie 2026

Rezumat

Această lucrare prezintă implementarea unei platforme web interactive destinată învățării și practicării programării competitive. Tema este relevantă deoarece folosește instrumente moderne de e-learning care să combată lipsa de motivare folosind tehnici de gamificare și interacțiune socială. Din punct de vedere tehnologic, lucrarea utilizează o arhitectură client-server bazată pe framework-ul Spring Boot pentru backend și o bază de date relațională gestionată prin Hibernate. Frontend-ul este scris în React cu TypeScript utilizând TanStack Query pentru administrarea eficientă a stărilor și Material UI pentru interfață. Rezultate obținute constă într-o platformă funcțională care integrează un motor de execuție automată a codului, un magazin de efecte vizuale și un modul social pentru interacțiunea utilizatorilor.

Abstract

This thesis presents the implementation of a web application used for learning and practising competitive programming. This topic is relevant because it uses modern e-learning methods to prevent the loss of motivation by using gamification and social interactions techniques. Technologically, the thesis uses a client-server architecture based on the Spring Boot framework on backend and a relational database managed using Hibernate. The frontend has been developed using React with TypeScript and TanStack Query for the efficient management of states and Material UI for interface. The obtained results consist of a fully functional platform that integrates an automated code execution engine, a shop for visual effects and a social module for user interaction.

Cuprins

Capitolul 1

Introducere

1.1 Motivația lucrării

Platformele tradiționale de programare se concentrează exclusiv pe rezolvarea unor probleme lungi și complexe. Această abordare devine adesea plictisitoare, anostă și solicitantă din punct de vedere cognitiv, ducând la abandon din partea utilizatorilor. Pentru a combate această monotonie, platforma propune o diversificare a dinamicii de învățare prin introducerea unui sistem mixt. Alternarea între scrierea de cod și rezolvarea unor chestionare îmbinate cu o componentă de comunitate menține motivația utilizatorului de a continua învățarea.

1.2 Obiectivele platformei

Principalul obiectiv al lucrării este implementarea unei platforme web interactive care îmbină execuția sigură a codului cu tehnici de gamificare și inteligență artificială. Pentru atingerea scopului, au fost stabilite următoarele obiective:

- Automatizarea generării de conținut: Integrarea modelelor de limbaj (Large Language Models) prin intermediul Ollama și Qwen2.5:coder permite generarea dinamică a chestionarelor zilnice și a distractorilor (variante greșite, dar plauzibile).
- Evaluarea securizată și deterministă a codului: Implementarea unui motor de eva-

luare bazat pe medii izolate (integrarea cu Judge0) care să permită rularea sigură și validarea precisă a algoritmilor scriși de utilizatori în C++ și Python.

- Retenția utilizatorilor prin gamificare și mecanisme sociale: Creare unei economii virtuale (sistem de experiență, monede, clasamente) și dezvoltarea unor mecanisme de interacțiune socială, precum seriile individuale și de grup sau de interacțiune pe pagina de postări.
- Asigurarea calității și scalabilității conținutului: Dezvoltarea unui modul dedicat creatorilor și moderatorilor, cu suport pentru formatare Markdown, scrierea de scripturi customizate în Python pentru validarea output-urilor multiple și gestionarea eficientă a raportărilor conținutului din cadrul comunității.

1.3 Structura lucrării

Lucrarea este împărțită în 5 capitole, organizate după cum urmează:

- Capitolul 1 – Introducerea. Definește contextul general al proiectului, prezentând motivația din spatele alegerii temei, obiectivele principale ale platformei și structura de ansamblu a documentului
- Capitolul 2 – Preliminarii. Abordează conceptele fundamentale ale domeniului (programare competitivă și gamificarea) și justifică alegerea stivei tehnologice utilizate pentru dezvoltarea componentelor de frontend și backend.
- Capitolul 3 – Analiza și proiectarea sistemului. Detaliază aspectele de arhitectură software, descriind structura de ansamblu client-server decuplată și modelul conceptual al datelor prin intermediul diagramei entitate-relație.
- Capitolul 4 – Implementarea sistemului. Reprezintă nucleul tehnic al lucrării și descrie în detaliu funcționalitățile dezvoltate: mecanismul de generare automată a chestionarelor prin LLM și validarea lor prin motorul de execuție Judge0, sisteme de gamificare și serii (streaks), modulul social și cel de moderare a conținutului.

- Capitolul 5 – Concluzii. Sintetizează modul de realizare a temei, evidențiază relevanța rezultatelor obținute, aplicabilitatea practică a platformei și propune direcții viitoare pentru extinderea și optimizarea aplicației.

Capitolul 2

Preliminarii

2.1 Programare competitivă

Programarea competitivă își are rădăcinile în studiul științific al algoritmilor. În timp ce un informatician scrie o demonstrație să arate că algoritmul său funcționează, un programator competitiv implementează acel algoritm și îl testează într-un sistem de competiții [3]. Aceasta reprezintă o modalitate de a învăța noi algoritmi și structuri de date, dar și tehnici diferite de optimizare și avantajele sau dezavantajele fiecăreia. De asemenea, aceasta ajută programatorii să se gândească la diferite soluții în funcție de contextul și restricțiile avute.

În ultimii ani, programarea competitivă a devenit un instrument important pentru dezvoltarea competențelor algoritmice și pentru pregătirea interviurilor tehnice din industria software. Numeroase companii din domeniul tehnologiei utilizează probleme algoritmice în procesele de recrutare pentru un prim filtru al candidaților, iar platformele dedicate permit utilizatorilor să își exerseze cunoștințele de structuri de date, algoritmi și optimizare.

Un element specific platformelor de programare competitivă îl reprezintă sistemul de evaluare automată a unei soluții. Acesta primește codul sursă transmis de utilizator, îl compilează și îl execută într-un mediu izolat, verificând dacă soluția a fost corectă prin rularea acesteia pe mai multe seturi de date de test. Acest mecanism permite evaluarea obiectivă și rapidă a unei soluții pentru o problemă propusă.

Proiectul prezentat în această lucrare se încadrează în categoria platformelor de programare competitivă, însă adaugă noi mecanisme prin integrarea unor elemente de gamificare, funcționalități sociale și generare automată de conținut educațional folosind inteligența artificială.

2.2 Gamificare

Gamificarea presupune aplicarea principiilor de design al jocului în procesul educațional, cu scopul de a face învățarea mai captivantă, motivantă și eficientă [1]. În cadrul acestui proiect, gamificarea este implementată prin următoarele mecanisme:

- Puncte de experiență: necesare pentru avansarea în nivel.
- Serii de activitate (streaks): menținute atât individual, cât și în colaborare cu alți utilizatori de pe platformă.
- Clasament: primii 10 utilizatori, clasati în funcție de experiența acumulată, sunt recompensați lunar cu monede virtuale.
- Recompense zilnice: un sistem prin care utilizatorii primesc bonusuri pentru activitatea constantă.

2.3 Tehnologii utilizate

2.3.1 Frontend

Componenta de client a platformei a fost dezvoltată în React, utilizând limbajul TypeScript. React este o bibliotecă open-source ce permite programatorului să construiască interfețe văzute de utilizator în module individuale numite componente. TypeScript extinde limbajul de programare JavaScript prin adăugarea unui sistem static de tipuri de date, facilitând detectarea erorilor în faza de dezvoltare și scrierea unui cod mai robust [5].

Pentru stilizarea interfeței a fost folosită biblioteca Material UI (MUI). Aceasta oferă componente de React predefinite care implementează specificațiile sistemului vizual Google Material Design [4]. Personalizarea suplimentară a componentelor și a structurii vizuale s-a realizat prin intermediul limbajului standard de stilizare CSS (Cascading Style Sheets) [6].

2.3.2 Backend

Componenta de server a platformei a fost dezvoltată în limbajul de programare Java utilizând biblioteca Spring Boot. Java a fost ales datorită robusteții sale, a ecosistemului matur și a paradigmei orientate pe obiecte, fiind un standard recunoscut pentru dezvoltarea aplicațiilor de tip enterprise scalabile. Utilizarea bibliotecii Spring Boot a eficientizat considerabil procesul de dezvoltare prin mecanismele sale de auto-configurare și injectare a dependențelor (Dependency Injection). Prin intermediul acestuia, componenta de backend expune un API (Application Programming Interface) de tip RESTful, permițând comunicarea cu aplicația client prin intermediul formatului de date JSON.

2.3.3 Baza de date

Pentru stocarea și gestionarea datelor a fost ales sistemul de gestiune a bazelor de date relaționale (SGBD) PostgreSQL. Persistența datelor este realizată prin intermediul Hibernate, o bibliotecă de tip ORM (Object-Relational Mapping), integrat în aplicație prin Spring Data JPA.

Acest mecanism permite asocierea directă a obiectelor din mediul de programare orientat pe obiecte (entitățile) în tabele relaționale. Pe baza acestor entități, schema bazei de date este generată automat la rulare. De asemenea, instrumentul ORM gestionează crearea tabelelor asociative necesare pentru modelarea relațiilor complexe, cum ar fi stocarea submisilor sau cererile de prietenie.

2.3.4 Judge0

O componentă importantă pentru toate platformele de programare competitivă o reprezintă posibilitatea de a compila și rula codul sursă într-un mediu izolat și sigur (de tip sandbox). Pentru acest mecanism proiectul integrează Judge0, un motor de execuție a codului robust, scalabil și cu suport pentru multiple limbaje de programare [2], care a fost instanțiat local.

Pentru a evalua o soluție, componenta de backend (controller-ul) apelează API-ul expus de Judge0 printr-o cerere de tip POST. Corpul cererii (payload-ul) conține: codul sursă, datele de intrare, rezultatul așteptat, codul de identificare al limbajelor C++ sau Python, limita de memorie de 12.800 KB și o limită de timp de 2 secunde.

2.3.5 Modele generative de limbaj

Proiectul integrează inteligența artificială pentru a crea automată de chestionare (quiz-uri). Această funcționalitate este realizată prin intermediul unui model de limbaj de mari dimensiuni (LLM – Large Language Model), rulat local cu ajutorul mediului de execuție Ollama. A fost selectat modelul Qwen2.5-coder, în varianta cu 7 miliarde de parametri.

Rolul acestuia este de a formula secvențe scurte de cod pe un subiect extras aleatoriu dintr-o listă predefinită, precum și de a concepe variante de răspuns incorecte, dar plauzibile (distractori). Deoarece execuția are loc local, calitatea și complexitatea textului rezultat sunt direct constrânse de dimensiunea relativ redusă a parametrilor modelului utilizat.

Capitolul 3

Analiza și proiectarea sistemului

3.1 Arhitectura de ansamblu

Pentru a asigura modularitatea, mentenanța facilă și securitatea resurselor, platforma a fost proiectată pe baza unei arhitecturi client-server decuplate. Server-ul din backend preia rolul de orchestrator central, gestionând regulile de business și coordonând serviciile specializate de execuție de cod și inteligență artificială.

Sistemul este divizat în patru straturi distincte, interconectate prin protocoale standard de rețea:

1. Nivelul de prezentare (Frontend): Implementat în React și TypeScript, responsabil de interacțiunea directă cu utilizatorul, afișarea enunțurilor în Markdown și capturarea soluțiilor la problemele algoritmice.
2. Nivelul de orchestrare și logică (Backend): Dezvoltat în Spring Boot, centralizează logica aplicației, autentificarea securizată, gestiunea economiei virtuale (experiență, monede), resetarea provocărilor zilnice și a clasamentului lunar și autorizarea pe bază de roluri (utilizator, creator, moderator și administrator).
3. Nivelul de persistență (Baza de date): Realizat în PostgreSQL pentru stocarea stării globale a sistemului: utilizatori, probleme de algoritmică, chestionare, submisii, relații de prietenie, mesaje, notificări.

4. Nivelul serviciilor specializate (Sandbox și LLM): Compus din motorul local Judge0 pentru compilarea și executarea izolată a codului și mediul Ollama (Qwen2.5-coder) pentru generarea chestionarelor și a răspunsurilor greșite.

O ilustrare grafică a acestor componente se regăsește în Figura 3.1.

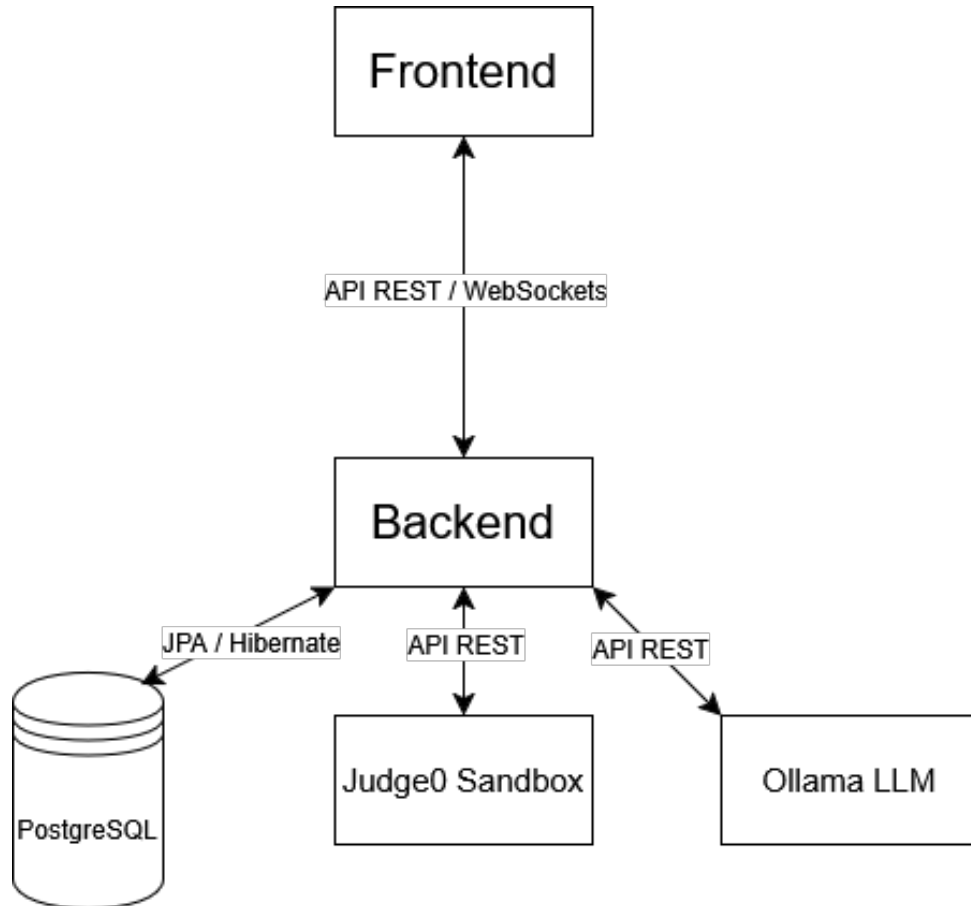


Figura 3.1: Arhitectura de ansamblu a platformei

3.2 Modelul conceptual al datelor

Persistența datelor este structurată relațional în PostgreSQL, fiind optimizată pentru a suporta interogări frecvente și relații complexe între modulele sociale și cele algoritmice. Structura bazei de date este reprezentată de următoarele entități nucleu:

- Account: stochează credențialele, hash-ul parolei, rolul de acces (utilizator, creator, moderator, administrator), nivelul de experiență, balanța de monede și detaliile de profil.

- Problem și Submission: modelează ierarhia problemelor algoritmice (incluzând metadate precum dificultatea și vizibilitatea) și stochează istoricul încercărilor de rezolvare trimise de utilizatori către motorul de evaluare.
- Quiz: păstrează structura chestionarelor generate automat, incluzând secvența de cod, limbajul (C++ sau Python), răspunsul corect și variantele de răspuns plauzibile.
- Friendship, Message și Post: gestionează rețeaua socială a platformei, de la stabilirea conexiunilor între conturi, până la schimbul de mesaje directe și publicarea de conținut.
- Streak: monitorizează activitatea utilizatorului, contorizând atât seriile zilnice individuale (prin rezolvarea problemei zilei), cât și pe cele colaborative dintre prieteni (prin rezolvarea chestionarului zilnic)

Pentru a menține performanța bazei de date și a evita timpii mari de răspuns, fișierele masive (scripturile de validare și fișierele cu seturile de date de intrare și ieșire) nu sunt stocate intern ca obiecte binare de mari dimensiuni (BLOB – Binary Large Object). Persistența lor se realizează în sistemul de fișiere al serverului, în timp ce tabelele stochează căile către aceste locații pe disc.

Ilustrarea grafică a acestor entități se află în Figura 3.2.

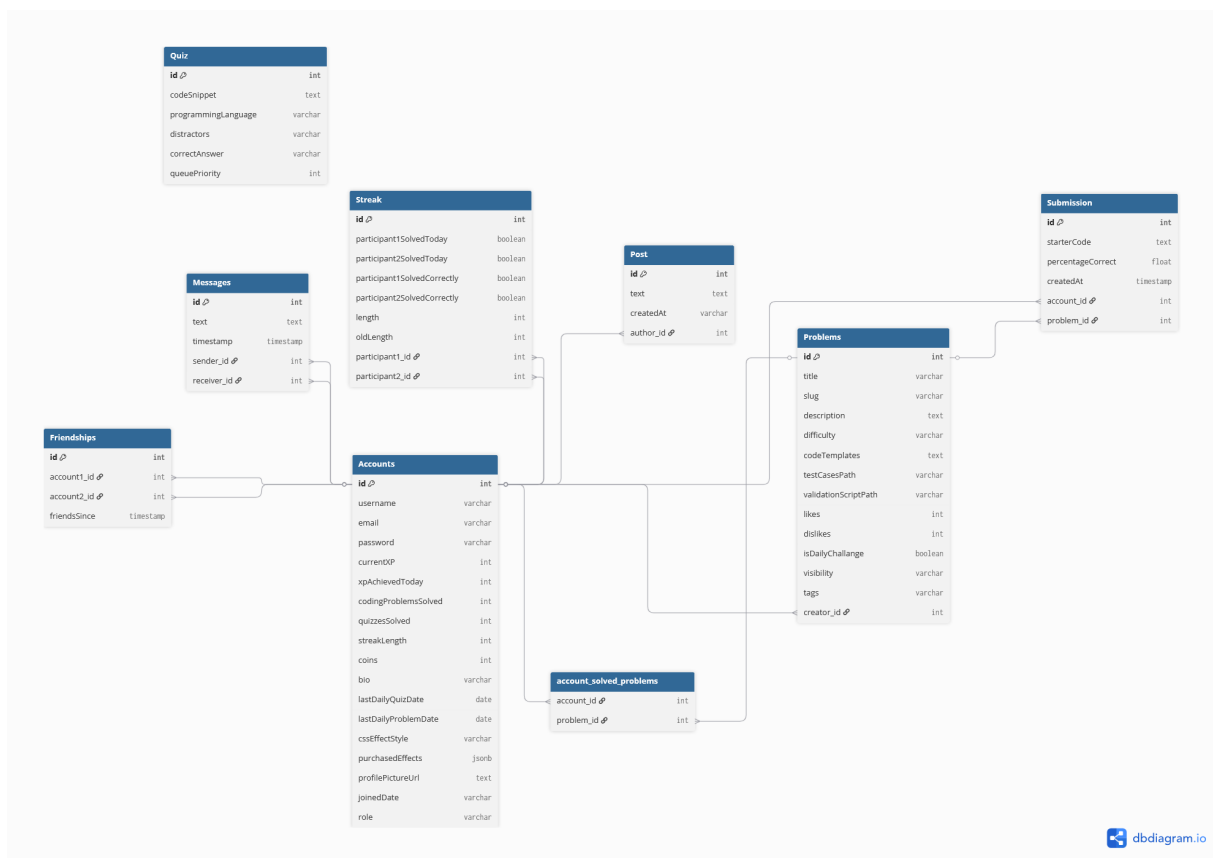


Figura 3.2: Arhitectura de ansamblu a platformei

Capitolul 4

Implementarea sistemului

4.1 Sistemul de autentificare

Accesul la funcționalitățile platformei este securizat printr-un modul de identificare, autentificare și autorizare a utilizatorilor, structurat pe două fluxuri: înregistrarea și conectarea.

4.1.1 Înregistrarea

Crearea unui cont necesită furnizarea unui nume de utilizator, adresă de email și a unei parole. Pentru integritatea sistemului, validarea datelor se realizează atât pe partea de client, cât și pe server, utilizatorul primind erori dacă informațiile introduse sunt necorespunzătoare. Numele este folosit în platformă pentru afișarea contului de utilizator, iar email-ul este folosit pentru recuperarea contului în cazul în care parola a fost uitată. Un nume de utilizator este valid dacă lungimea este între 5 și 20 de caractere alfanumerice. Validarea email-ului se face prin librăria React Hook Form folosind expresii regulate, iar un email este valid dacă respectă șablonul `/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/`. Validarea parolei constă în verificarea lungimii acesteia și este validă dacă are cel puțin 6 caractere și cel mult 100 de caractere.

Pentru securizarea confidențialității, parolele nu sunt stocate în text clar. Înainte de stocare, parola este trecută printr-o funcție de hashing criptografic folosind algoritmul

Bcrypt, generându-se un hash ireversibil ce include automat un salt pentru a preveni atacurile de tip rainbow table.

4.1.2 Conectarea

Pentru conectare este necesară furnizarea unei adrese de email și parole corespunzătoare contului. Parola introdusă este validată prin compararea hash-ului cu cel stocat. În caz de succes, sistemul generează un token de tip JWT (Json Web Token) folosit pentru menținerea stării sesiunii (stateless). Pentru a mitiga riscurile atacurilor de tip XSS (Cross-Site Scripting) și furtul de token-uri, JWT-ul nu este stocat în localStorage, ci este transmis și salvat exclusiv în cookie-uri securizate prin flag-ul HttpOnly.

În scenariul pierderii accesului, platforma dispune de un mecanism de recuperare a contului. La solicitarea utilizatorului, sistemul generează un token unic cu o valabilitate limitată la 15 minute, care este expediat sub forma unui link securizat către adresa de email asociată.

4.2 Sistemul de gestionare a problemelor algoritmice

4.2.1 Crearea și stocarea problemelor

Platforma dispune de un mecanism prin care utilizatorii cu rolul de creator pot crea probleme de programare care pot fi rezolvate. Definirea enunțului se realizează prin intermediul unui editor Markdown cu suport pentru previzualizare. La nivel arhitectural, sursa unei probleme este decuplată în două componente distincte pentru ambele limbaje suportate (C++ și Python): Driver code (modulul ascuns responsabil de apelarea funcțiilor și evaluarea performanței) și Starter code (scheletul de cod expus utilizatorului). Integrarea celor două componente se realizează printr-un proces de injecție a codului, utilizând token-ul specific `{{CODE}}` în cadrul Driver code-ului.

Pentru evaluare, sistemul permite crearea unui număr nelimitat de cazuri de testare (perechi de intrare și ieșire). Pentru a optimiza performanța bazei de date, testele nu sunt salvate ca obiecte, ci în baza de date va fi stocat doar calea la care se regăsesc acele teste.

Entitatea problemei este completată de metadate esențiale indexării: denumire, nivel de dificultate și etichete algoritmice.

4.2.2 Sistemul de validare a unei soluții

Pentru scenariul în care o problemă de programare poate avea mai multe soluții corecte pentru aceleași date de intrare, platforma implementează un sistem de evaluare flexibil, bazat pe scripturi de validare. Creatorul poate să scrie un script de evaluare exclusiv în limbajul Python acesta urmând a fi rulat automat la fiecare test.

Comunicarea dintre mediul de evaluare și scriptul de validare se realizează prin trecerea datelor de intrare originale și a rezultatului generat de utilizator, separate prin delimitatorul `@@@USER_OUTPUT@@@`. Scriptul analizează dacă rezultatul produs de soluția utilizatorului este valid, iar în caz afirmativ va trebui afișat la ieșirea standard mesajul `True` sau `False` în caz contrar. Similar testelor, acest script nu este salvat în baza de date, ci doar locația acestuia pe disc și este executat pentru fiecare caz de testare în parte.

4.2.3 Publicarea problemelor

Pentru a asigura calitatea conținutului, orice problemă nou creată este inițializată cu un status privat (modul de vizibilitate ascuns). Această stare izolează problema de mediul de producție, permițând creatorului să o testeze prin motorul de execuție pentru a detecta eventuale erori de logică. Ulterior fazei de testare și depanare, creatorul poate seta vizibilitatea problemei ca public urmând să fie expusă în lista de probleme disponibile de pe platformă.

4.3 Sistemul de chestionare generate automat

Platforma dispune de un mecanism interactiv denumit „Chestionarul zilei”, prin care utilizatorilor li se cere să identifice rezultatul corect (output-ul) al unei secvențe de cod generate aleatoriu scrisă în C++ sau Python. Chestionarul conține o singură variantă

corectă și trei variante incorecte (distractori).

Generarea conținutului este automatizată prin intermediul modelului de limbaj Qwen2.5-coder (7 miliarde de parametri), apelat prin API-ul mediului Ollama.

4.3.1 Fluxul de generare și validare

Crearea unui chestionar complet presupune un proces în două etape pentru a garanta corectitudinea tehnică:

1. Generarea codului sursă: Backend-ul transmite un prompt în limba engleză către modelul LLM, solicitând un cod determinist, concis (maximum 40-50 de linii) și valid. Subiectele sunt alese aleatoriu din liste specifice:
 - C++: Programare orientată pe obiecte (OOP), pointeri, referințe, algoritmi, excepții.
 - Python: OOP, list comprehensions, generatori, decoratori, expresii regulate, algoritmi.
2. Validarea și generarea distractorilor: Codul rezultat este trimis către motorul de execuție Judge0 pentru compilare și execuție:
 - Dacă execuția are succes, secvența și rezultatul ei sunt retrimise către LLM pentru a genera trei variante de răspuns greșite, dar plauzibile (distractori)
 - Dacă secvența nu compilează, sistemul declanșează automat până la două re-încercări. În cazul în care toate încercările eșuează, server-ul returnează fie o eroare, fie chestionarele generate cu succes până la acel moment (în cazul generării în masă).

Exemplu de structură a prompt-ului transmis modelului: „Ești un dezvoltator expert în [limbaj de programare]. Creează un program complet, executabil, care demonstrează [un subiect aleator]. Cerințe: comportament determinist, concis, entități declarate corect, fără comentarii, returnează exact o linie de output. Răspunsul trebuie să fie strict în format JSON.”

4.3.2 Administrarea chestionarelor

Această funcționalitate este controlată din panoul de administrare („Manage quizzes”), fiind accesibilă doar moderatorilor și administratorilor. Aceștia pot genera chestionare individual (pentru a corecta eventualele halucinații sau răspunsuri nedeterminate ale modelului) sau în masă.

Chestionarele generate sunt stocate într-o listă și pot fi reordonate manual de către administratori pentru a asigura diversitatea zilnică. Sistemul selectează automat chestionarul cu cea mai mare prioritate (prima poziție) pentru ziua curentă, ștergându-l din baza de date la finalul zilei.

4.3.3 Sistemul de recompense

Rezolvarea corectă a chestionarului zilei aduce utilizatorilor puncte de experiență (pentru clasament) și monede virtuale. Sistemul încurajează consecvența prin menținerea unei serii de activitate (streak), inclusiv în format colaborativ cu prietenii.

Dacă un utilizator ratează o zi sau răspunde greșit, seria întregului grup este resetată. Totuși, platforma oferă opțiunea de a salva seria („streak freeze”) în schimbul a 200 de monede, valabilă exclusiv în cazul unui răspuns greșit.

Atingerea anumitor praguri în menținerea seriei zilnice este recompensată cu bonusuri în monede, după cum urmează:

- 7 zile: 20 de monede
- 14 zile: 50 de monede
- 30 zile: 150 de monede
- 100 zile: 500 de monede
- Peste 100 zile: 550 monede (acordate la fiecare 30 de zile suplimentare)

4.4 Sistemul de gamificare

4.4.1 Experiența și rangurile

Experiența (XP) reprezintă metrica principală pentru cuantificarea activității utilizatorilor și stabilirea poziției acestora în clasament. Punctele de experiență pot fi acumulate prin rezolvarea chestionarului zilei, a problemei zilei, precum și prin rezolvarea altor probleme algoritmice disponibile pe platformă.

Acumularea de experiență determină creșterea în nivel a contului. Pentru a reflecta tematica platformei, rangurile sunt denumite pe baza unităților de măsură a informației. Fiecărui rang îi este asociată o culoare distinctă pentru identificarea vizuală rapidă pe profilul utilizatorilor:

- 0 - 199 XP: Bit (gri)
- 200 - 599 XP: Byte (verde)
- 600 - 1499 XP: Kilobyte (turcoaz)
- 1500 - 2999 XP: Megabyte (mov)
- 3000 - 5999 XP: Gigabyte (portocaliu)
- Peste 6000 XP: Terabyte (roșu)

4.4.2 Monedele virtuale

Monedele reprezintă valuta internă a platformei. Acestea se obțin în urma menținerii seriilor zilnice (streaks) alături de prieteni, rezolvării provocărilor zilnice (chestionar și problemă) și a clasării în top 10 la finalul lunii.

Utilizatorii pot folosi monedele în magazinul virtual al platformei pentru:

- Salvarea seriei cu prietenii („streak freeze”): costă 200 de monede și previne pierderea progresului zilnic în cadrul chestionarelor.

- Efecte vizuale pentru avatar: personalizarea profilului cu efecte vizibile global (Fire effect - 1.000 monede, Ice effect - 1.500 monede, Lightning effect - 2.000 monede, Quantum Pulse effect - 3.000 monede).

4.4.3 Clasamentul

Pentru a stimula competitivitatea, platforma integrează un clasament bazat pe experiența acumulată. Interfața evidențiază utilizatorii cu cele mai bune performanțe din top.

Clasamentul are un caracter recurent, fiind resetat în ultima zi a fiecărei luni. Odată cu această resetare, punctele de experiență și nivelul utilizatorilor revin la zero, iar performanțele de top sunt recompensate automat cu monede, după cum urmează:

- Locul 1: 500 de monede
- Locul 2 și 3: 250 de monede
- Locurile 4 - 10: câte 100 de monede

4.5 Sistemul de serii

Seriile de activitate (streaks) sunt utilizate pentru a monitoriza și stimula consecvența utilizatorilor pe platformă. Sistemul este împărțit în două categorii distincte: o serie individuală pentru rezolvarea problemelor și o serie colaborativă pentru rezolvarea chestionarelor.

4.5.1 Seria individuală (problema zilei)

Fiecare rezolvare succesivă a problemei zilei crește seria individuală a utilizatorului cu o zi. Această serie are un rol pur onorific și statistic, nefiind asociată cu recompense în monede la atingerea anumitor praguri.

4.5.2 Seria colaborativă (chestionarul zilei)

Această serie se bazează pe interacțiunea socială și se menține în pereche cu un prieten de pe platformă. Regulile de evoluție a seriei sunt următoarele:

- Creșterea seriei: condiția de bază pentru ca seria să avanseze cu o zi este ca ambii utilizatori să completeze chestionarul în aceeași zi și să răspundă corect.
- Resetarea seriei: dacă ambii utilizatori oferă un răspuns greșit sau dacă cel puțin unul dintre ei ratează completarea chestionarului în intervalul de 24 de ore, seria comună se resetează la 0.
- Salvarea seriei: în cazul în care un utilizator completează chestionarul, dar răspunde greșit, acesta are posibilitatea exclusivă de a-și salva seria prin achitarea unei taxe de 200 de monede virtuale („streak freeze”). Opțiunea de salvare nu este disponibilă dacă ziua a fost ratată complet.

4.6 Sistemul de interacțiune socială

Platforma încorporează o componentă de interacțiune socială între utilizatori. Această componentă este alcătuită din trei module: gestionarea relațiilor de prietenie, fluxul de postări și serviciul de mesagerie privată.

4.6.1 Relațiile de prietenie

Utilizatorii pot căuta alte conturi în cadrul platformei pe baza numelui de utilizator (username) și pot trimite o cerere de prietenie. Trimiterea unei solicitări generează o notificare către contul țintă. Dacă destinatarul acceptă, relația de prietenie este stabilită, astfel aceasta este respinsă, iar expeditorul nu este notificat de decizia respingerii cererii.

Pentru optimizarea performanței, căutarea conturilor folosește o paginare bazată pe cursor (cursor-based pagination), API-ul returnând seturi de câte 20 de rezultate. De asemenea, interogarea bazei de date nu face distincție între majuscule și minuscule (case insensitive).

4.6.2 Fluxul de postări

Platforma permite utilizatorilor să publice conținut în cadrul unui flux dedicat (feed), postările fiind vizibile exclusiv utilizatorilor din lista de prieteni. O postare poate include un mesaj text și atașamente în diverse formate, cum ar fi: imagini, fișiere PDF, fișiere text simple (.txt) și documente Word (.doc, .docx).

Prietenii autorului au posibilitatea de a vizualiza aceste postări, de a descărca fișierele atașate și de a adăuga comentarii. Sistemul de comentarii este identic din punct de vedere funcțional, oferind suport atât pentru introducerea de text, cât și pentru atașarea de fișiere în aceleași formate menționate anterior.

4.6.3 Mesagerie privată

Comunicarea directă și privată este restricționată, fiind permisă exclusiv între utilizatorii care au stabilit o relație de prietenie. Mesajele text acceptă aceleași tipuri de atașamente menționate în cadrul modului de postări.

Pentru a asigura transmiterea și afișarea mesajelor în timp real (fără reîncărcarea paginii), canalul de comunicare dintre client și server este implementat prin intermediul protocolului WebSocket.

4.7 Sistemul de moderare a conținutului

Pentru a asigura un mediu digital sigur și a preveni abuzurile, platforma integrează un modul dedicat raportării și moderării conținutului.

4.7.1 Mecanismul de raportare și soluționare

Utilizatorii au posibilitatea de a semnală nereguli legate de comportamentul altor conturi sau de conținutul publicat. Entitățile care pot fi raportate includ: problemele de programare, postările, comentariile, mesajele private și conturile de utilizator.

Soluționarea acestor sesizări este realizată de către moderatori sau administratori, care au la dispoziție următoarele acțiuni:

- Ignorarea raportului: în cazul în care sesizarea este eronată sau nefondată.
- Gestionarea conținutului: editarea sau eliminarea problemelor de programare neconforme, ștergerea mesajelor, postărilor sau a comentariilor neadecvate.
- Sanționarea utilizatorului: suspendarea definitivă (ștergerea) contului în caz de abateri grave.

4.7.2 Gestionarea drepturilor de acces

Acordarea rolurilor speciale de creator sau moderator este o permisiune exclusivă a administratorilor. În momentul în care unui utilizator îi sunt modificate privilegiile, acesta va primi o notificare automată.

Din punct de vedere tehnic, pentru aplicarea noilor permisiuni, utilizatorul trebuie să se reconecteze. Acest proces este obligatoriu pentru a invalida token-ul de autentificare curent și a genera unul nou, care să conțină drepturile de acces actualizate.

Capitolul 5

Concluzii

În cadrul acestei lucrări, obiectivele propuse au fost îndeplinite cu succes prin dezvoltarea unei platforme web complexe de programare competitivă. Proiectul a demonstrat viabilitatea integrării unei arhitecturi client-server utilizând tehnologii moderne: React și TypeScript pentru interfață, respectiv Spring Boot și PostgreSQL pentru logica de backend și persistența datelor. Integrarea cu succes a motorului de execuție Judge0 și a modelelor generative de limbaj (Qwen2.5-coder) a permis automatizarea fluxurilor de evaluarea a codului și de generare a conținutului educațional.

Consider că rezultatele obținute sunt relevante în contextul educațional actual din domeniul IT. Învățarea algoritmilor este adesea percepută ca fiind rigidă și monotună, iar prin adăugarea unei componente solide de gamificare procesul este transformat într-o experiență interactivă și mai atractivă.

Deși aplicația este complet funcțională în stadiul curent, flexibilitatea arhitecturii permite extinderi viitoare de interes academic și tehnic:

- Extinderea suportului pentru LLM: migrarea către modele generative mai mari găzduite în cloud sau fine-tuning-ul unui model local dedicat exclusiv generării de probleme algoritmice complexe.
- Module de competiție în timp real: implementarea unui sistem de meciuri directe între utilizatori.
- Dezvoltarea unei aplicații mobile: transpunerea componentelor de gamificare (în

special mecanismul de chestionarul zilei) într-o aplicație mobilă pentru a facilita accesul rapid al utilizatorilor și a crește rata de retenție.

Bibliografie

- [1] S. Apetrei, „Joc, gamificare și învățarea prin gamificare”, în Revista cadrelor didactice 116 (Feb. 2025), [accesat: 31.05.2026], URL: <https://www.didactic.ro/revista-cadrelor-didactice/joc-gamificare-si-invatarea-prin-gamificare>.
- [2] H. Z. Došilović și I. Mekterović, „Robust and Scalable Online Code Execution System”, în 43rd International Convention on Information, Communication and Electronic Technology (MIPRO), 2020, DOI: [10.23919/MIPRO48935.2020.9245310](https://doi.org/10.23919/MIPRO48935.2020.9245310).
- [3] Antti Laaksonen, Guide to Competitive Programming, Cham: Springer, 2017, DOI: [10.1007/978-3-319-72547-5](https://doi.org/10.1007/978-3-319-72547-5).
- [4] MUI Core, Material UI: A popular React UI framework, [accesat: 31.05.2026], URL: <https://mui.com/material-ui/>.
- [5] TypeScript, TypeScript is JavaScript with syntax for types, [accesat: 31.05.2026], URL: <https://www.typescriptlang.org/>.
- [6] World Wide Web Consortium (W3C), Cascading Style Sheets (CSS), [accesat: 31.05.2026], URL: <https://www.w3.org/Style/CSS/>.